

DOKUMENTASI TEKNIS & ARSITEKTUR SISTEM

Inventory Management & Point of Sale System

Versi	1.0
Tanggal	Agustus 2026
Platform	Web Application
Stack	Next.js + NestJS + PostgreSQL

DAFTAR ISI

1. Executive Summary
2. Arsitektur Sistem
3. Technology Stack
4. Database Schema
5. Modul & Fitur
6. Alur Bisnis (Business Flow)
7. Security & Best Practices
8. API Endpoints
9. Deployment Guide
10. Panduan Pengembangan

1. Executive Summary

Sistem Inventory & POS ini dirancang sebagai solusi terintegrasi untuk manajemen gudang, distribusi barang ke outlet, dan proses penjualan (Point of Sale). Sistem mendukung multi-warehouse dan multi-outlet, dengan fitur tracking batch per produk menggunakan QR Code, manajemen expired date secara otomatis, serta integrasi payment gateway Midtrans.

Keunggulan Utama:

- Dynamic RBAC — Role dan permission diatur admin, menu menyesuaikan otomatis
- QR Code Tracking — Setiap batch produk punya QR unik dengan data expired
- FIFO Expired Management — Batch terlama terjual duluan, expired otomatis terblokir
- Multi-location — Mendukung banyak gudang dan outlet
- Split Payment — Tunai, Midtrans, dan kombinasi keduanya
- Realtime Stock — Pergerakan stok tercatat di setiap transaksi
- Flexible Schema — Database dirancang generic untuk berbagai jenis toko

2. Arsitektur Sistem

Sistem menggunakan arsitektur client-server dengan pemisahan frontend dan backend:

Layer	Technology	Deskripsi
Frontend	Next.js 14 + React 18	Dashboard admin, POS interface, SSR/CSR
Backend API	NestJS 10 + TypeScript	REST API, business logic, auth
Database	PostgreSQL 16	Primary data store, relational
Cache (opt)	Redis 7	Session, caching (future)
Payment	Midtrans Snap	Payment gateway integration

Arsitektur Diagram:



3. Technology Stack

Backend:

- NestJS 10 — Framework Node.js dengan arsitektur modular
- TypeORM — ORM untuk PostgreSQL dengan migration support
- Passport + JWT — Authentication dengan access/refresh token
- class-validator — Input validation dan sanitization
- Helmet — HTTP security headers
- @nestjs/throttler — Rate limiting
- @nestjs/schedule — Cron jobs (expired check)
- @nestjs/swagger — Auto-generated API documentation
- nestjs-pino — Structured logging

Frontend:

- Next.js 14 — React framework dengan App Router
- TypeScript — Type safety end-to-end
- Tailwind CSS 3 — Utility-first CSS framework
- Zustand — Lightweight state management
- Axios — HTTP client dengan interceptor
- Lucide React — Icon library
- Radix UI — Accessible headless components

4. Database Schema

Database dirancang dengan prinsip normalisasi dan flexibility. Menggunakan UUID sebagai primary key, soft delete, dan timestamp tracking.

Entity/Table	Modul	Deskripsi
users	Auth	Data user (email, password hash, profile)
refresh_tokens	Auth	JWT refresh token storage
roles	RBAC	Role definitions
permissions	RBAC	Permission definitions (module:action)
role_permissions	RBAC	Many-to-many role-permission
user_roles	RBAC	User-role assignments
modules	RBAC	Dynamic menu/module tree
warehouses	Master	Data gudang
outlets	Master	Data outlet/toko
categories	Master	Kategori produk (tree structure)
user_locations	Master	Assignment user ke lokasi
products	Inventory	Master produk (SKU, harga, atribut)
product_batches	Inventory	Batch per produk (expired, QR)
inventory	Inventory	Stok per lokasi per batch
stock_movements	Inventory	History pergerakan stok
purchase_orders	PO	Surat pengiriman barang
po_items	PO	Detail item dalam PO
transactions	POS	Transaksi penjualan
transaction_items	POS	Item dalam transaksi
payments	POS	Pembayaran (cash/midtrans)
members	POS	Data member/loyalty
point_transactions	POS	History point member
promos	POS	Promo & diskon engine
returns	POS	Retur barang
return_items	POS	Detail item retur
notifications	System	Notifikasi user

5. Modul & Fitur

Authentication

- Register, Login, Logout
- JWT Access Token (15 menit) + Refresh Token (7 hari)
- Password hashing dengan bcrypt (cost factor 12)
- Rate limiting pada endpoint auth

RBAC (Role-Based Access Control)

- Dynamic roles — dibuat dan diatur admin
- Permission format: module:action (e.g. inventory:read)
- Guard decorator `@RequirePermission('...')`
- Menu dashboard otomatis menyesuaikan permission user

Master Data

- CRUD Warehouses, Outlets, Categories
- Category tree structure (parent-child)
- User-location assignment (user hanya akses lokasi yang di-assign)

Inventory & Product

- CRUD Products dengan dynamic attributes (JSONB)
- Batch management (production date, expired date)
- QR Code generation per batch
- Stock tracking per lokasi per batch
- Stock movements logging
- Cron job daily: auto-block batch expired

Purchase Order (PO)

- Flow: Draft → Submitted → Shipped → Received → Approved
- Stock otomatis terpotong dari warehouse saat ship
- Stock otomatis bertambah di outlet saat approve
- Search by PO number (untuk scan/input di outlet)

POS (Point of Sale)

- Scan QR / search product → add to cart
- FIFO: batch paling dekat expired dijual duluan

- Block otomatis: batch expired tidak bisa dijual
- Diskon per item & per transaksi
- Hold & recall transaction
- Void transaction

Payment

- Cash payment (hitung kembalian)
- Midtrans integration (Snap token)
- Split payment (sebagian cash, sebagian digital)
- Webhook untuk notification dari Midtrans

Member & Loyalty

- Registrasi member (name, phone, email)
- Point earning: 1 point per Rp 10.000
- Tier system: Bronze, Silver, Gold, Platinum
- Point redemption (future enhancement)

Reports

- Sales report (harian/mingguan/bulanan/per outlet)
- Stock report & low stock alert
- Expired/waste report
- Profit/Loss per outlet
- Cashier recap
- Member report
- Export to Excel/PDF

Notifications

- Expired warning (H-7, H-3, H-1)
- Low stock alert
- PO status change notification

6. Alur Bisnis (Business Flow)

A. Alur Input Barang & Distribusi:

1. Staff gudang input produk baru ke sistem
2. Buat batch (set tanggal produksi & expired)
3. Sistem generate QR Code → print & tempel di barang
4. Stock In ke warehouse (stok bertambah)
5. Buat Purchase Order (pilih outlet tujuan, pilih batch)
6. Submit PO → Review → Ship (stok gudang berkurang)
7. Outlet scan/input nomor PO → detail muncul
8. Outlet verifikasi & approve → stok outlet bertambah

B. Alur Penjualan (POS):

1. Kasir scan QR produk atau search manual
2. Sistem lookup batch (FIFO) + check expired
3. Produk masuk ke cart → hitung subtotal
4. Input member (optional) → apply promo/diskon
5. Pilih metode bayar: Cash / Midtrans / Split
6. Proses pembayaran → stok berkurang → movement tercatat
7. Member dapat point (jika ada)
8. Cetak struk / kirim digital receipt

7. Security & Best Practices

Helmet — HTTP security headers (CSP, HSTS, X-Frame-Options, dll)

Rate Limiting — 100 req/ menit per IP (configurable)

CORS — Whitelist origin frontend only

JWT — Short-lived access token + long-lived refresh token

Bcrypt — Password hashing dengan cost factor 12

Validation — class-validator whitelist mode — reject unknown fields

Soft Delete — Data tidak pernah benar-benar dihapus

SQL Injection — TypeORM parameterized queries

XSS — React auto-escaping + CSP headers

Audit Trail — Stock movements, created_by fields

8. API Endpoints

Base URL: <http://localhost:3000/api/v1>
Swagger Docs: <http://localhost:3000/docs>

Method	Endpoint	Description
POST	/auth/register	Register user baru
POST	/auth/login	Login → access + refresh token
POST	/auth/refresh	Refresh access token
POST	/auth/logout	Revoke refresh token
GET	/rbac/roles	List semua roles
POST	/rbac/roles	Create role
GET	/rbac/permissions	List permissions
GET	/rbac/menu	Get menu sesuai user permission
CRUD	/master/warehouses	Manage warehouses
CRUD	/master/outlets	Manage outlets
CRUD	/master/categories	Manage categories
CRUD	/inventory/products	Manage products
POST	/inventory/batches	Create batch + generate QR
POST	/inventory/stock-in	Stock in ke warehouse
GET	/inventory/stock/:type/:id	Get stock by location
POST	/purchase-orders	Create PO
PUT	/purchase-orders/:id/ship	Ship PO
PUT	/purchase-orders/:id/receive	Receive PO
PUT	/purchase-orders/:id/approve	Approve PO
POST	/pos/transactions	Create transaction
POST	/pos/payment	Process payment
POST	/pos/members	Register member
GET	/notifications	Get user notifications

9. Deployment Guide

Development:

1. docker compose up -d (PostgreSQL + Redis)
2. cd be && cp .env.example .env && npm install
3. npm run start:dev (Backend di port 3000)
4. npm run seed (Seed admin user & permissions)
5. cd fe && npm install && npm run dev (Frontend di port 3001)
6. Buka <http://localhost:3001> → Login: admin@inventorypos.com / Admin@123

Production:

- Set NODE_ENV=production di .env
- Ganti JWT_SECRET dan JWT_REFRESH_SECRET dengan random secure key
- Set DB credentials production
- Frontend: npm run build → deploy ke Vercel/server
- Backend: npm run build → pm2 start dist/main.js
- Setup reverse proxy (Nginx) dengan HTTPS
- Disable synchronize di TypeORM, gunakan migrations

10. Panduan Pengembangan

Struktur Project:

```
inventory-pos/
  ■■■ docker-compose.yml
  ■■■ be/ (NestJS Backend)
  ■ ■■■ src/
  ■ ■ ■■■ main.ts
  ■ ■ ■■■ app.module.ts
  ■ ■ ■■■ config/ (DB, env validation)
  ■ ■ ■■■ common/ (BaseEntity, DTOs, filters)
  ■ ■ ■■■ health/ (Health check)
  ■ ■ ■■■ modules/
  ■ ■ ■■■ auth/ (JWT, login, register)
  ■ ■ ■■■ rbac/ (Roles, permissions, menu)
  ■ ■ ■■■ master/ (Warehouse, outlet, category)
  ■ ■ ■■■ inventory/ (Product, batch, stock)
  ■ ■ ■■■ purchase-order/
  ■ ■ ■■■ pos/ (Transaction, payment, member)
  ■ ■ ■■■ notifications/
  ■ ■■■ package.json
  ■■■ fe/ (Next.js Frontend)
  ■■■ src/
  ■ ■■■ app/ (Pages - App Router)
  ■ ■■■ components/ (UI components)
  ■ ■■■ lib/ (API client, utils)
  ■ ■■■ store/ (Zustand stores)
  ■ ■■■ types/ (TypeScript interfaces)
  ■■■ package.json
```